

T2 TECHNOLOGIES

Feeder Balancing Module (FBM) Tool — Tender Enquiry 3239CXMWP

Author work package — Thabiso

This brief consolidates every section of the tender submission draft assigned to Thabiso, with the current draft content reproduced in full so each section can be developed independently before being merged back into the master document.

Assignment Summary

Section	Title	Ownership
5	Solution Architecture and Design Principles	Owned by Thabiso
6	Delivery Methodology and Implementation Plan	Owned by Thabiso
7	Programme Governance and Quality Management	Owned by Thabiso
17	Commercial Proposal Framework / Pricing Schedule	Shared — Tumelo / Thabiso

5. Solution Architecture and Design Principles

Owned by Thabiso

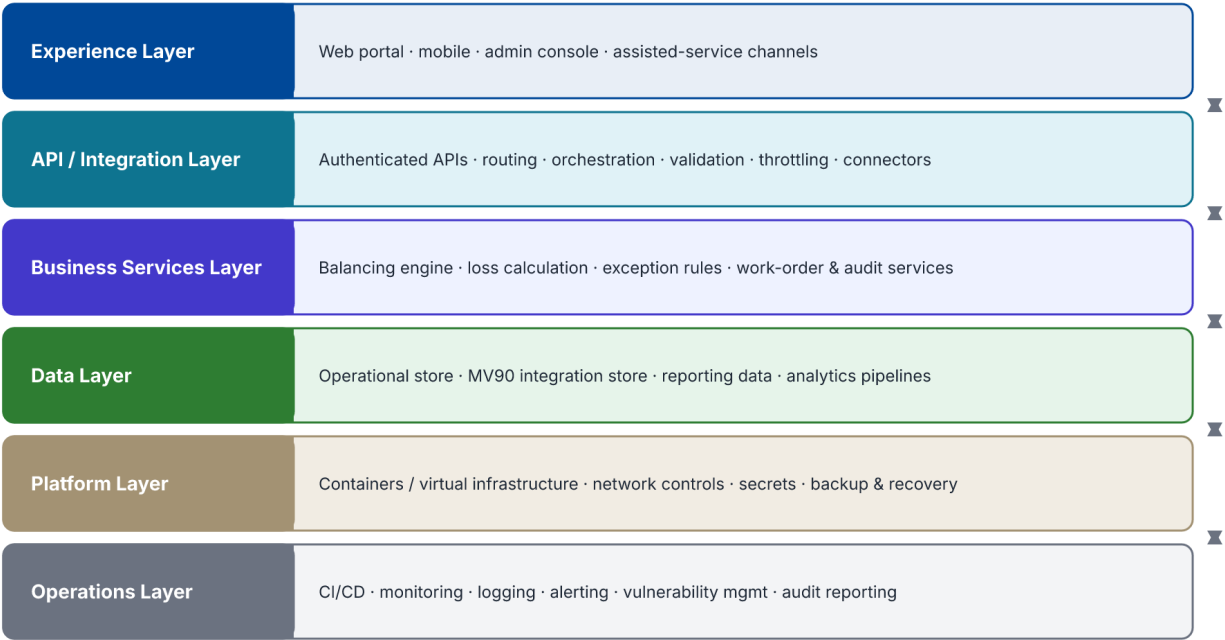
The target architecture will be confirmed during discovery, but T2 Technologies will apply the following principles unless the Client's standards require otherwise.

Principle	Application
Modularity	Separate business capabilities into clear modules and interfaces so that components can evolve independently.
API First	Expose controlled interfaces for system interaction instead of direct database coupling wherever practicable.
Security by Design	Apply identity, authorisation, encryption, secrets management, auditability and secure coding from the start.
Traceability	Use correlation identifiers and structured logs to trace transactions across service boundaries.
Resilience	Apply timeouts, retries where safe, circuit-breaking or isolation patterns, failure handling and recovery controls.
Idempotency	Protect critical write operations from accidental duplication when business or integration flows can be retried.
Observability	Capture metrics, logs and service health to support proactive operational management.
Data Governance	Define ownership, quality controls, retention, access and lineage for critical data.
Automation	Automate builds, tests, deployments and repeatable infrastructure tasks to reduce operational error.
Portability	Avoid unnecessary technology lock-in and preserve the ability to evolve the deployment model.

5.1 Illustrative Logical Architecture

- Experience Layer: web portals, mobile applications, administration interfaces and assisted-service channels.
- API / Integration Layer: authenticated APIs, routing, orchestration, validation, throttling, transaction controls and external connectors.
- Business Services Layer: domain-specific services, workflows and business rules.
- Data Layer: operational databases, integration stores, reporting data and controlled analytics pipelines.
- Platform Layer: runtime environments, containers/virtual infrastructure, network controls, secrets, backup and recovery.
- Operations Layer: CI/CD, monitoring, logging, alerting, vulnerability management, service management and audit reporting.

FBM Logical Architecture — layered view



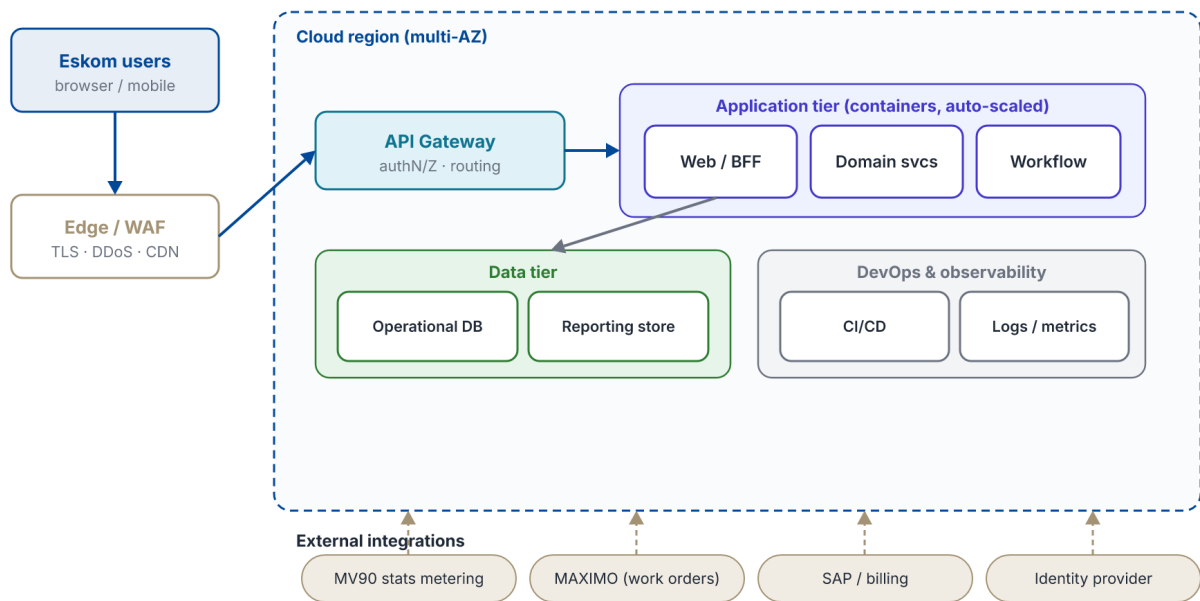
Each layer depends only on the layer directly beneath it; interfaces are controlled and versioned.

Figure 5.1 — Logical architecture

5.2 Reference Deployment and Data Flow

The module is designed for a resilient, cloud-ready deployment. Traffic is terminated at a managed edge, routed to stateless application services behind a private network boundary, and backed by managed data and integration stores. The data-flow view shows how metered readings move through import, validation and balancing into the decisions and actions the tool surfaces.

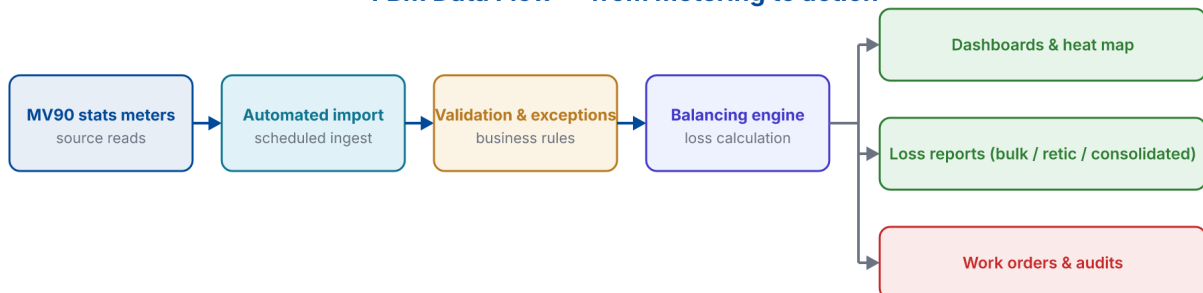
FBM Deployment Architecture — cloud reference



Secrets, encryption in transit and at rest, and role-based access apply across every tier.

Figure 5.2 — Deployment architecture

FBM Data Flow — from metering to action



Every stage is logged with correlation identifiers for full transaction traceability.

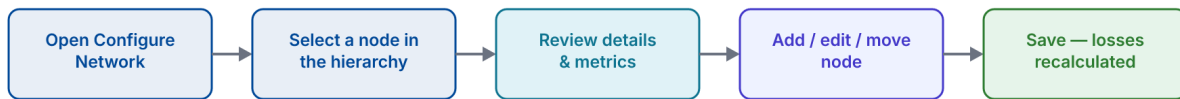
Figure 5.3 — Data flow

5.3 Application Configuration Workflows

The following workflows explain how the tool is operated day to day — behaviour that cannot be conveyed by a static screen alone. Each workflow diagram is paired with a capture of the corresponding screen in the live FBM application.

Configuring a network establishes the eight-level hierarchy (Operating Unit down to Meter); network mapping then lets an engineer re-parent equipment and group meters, transformers or feeders into clusters by drag-and-drop, with energy roll-ups recalculated automatically.

Workflow — Configuring the network hierarchy



Placement rules are enforced on save (e.g. a meter must sit under a transformer in the same substation), preventing invalid topologies.

Figure 5.4 — Network configuration workflow

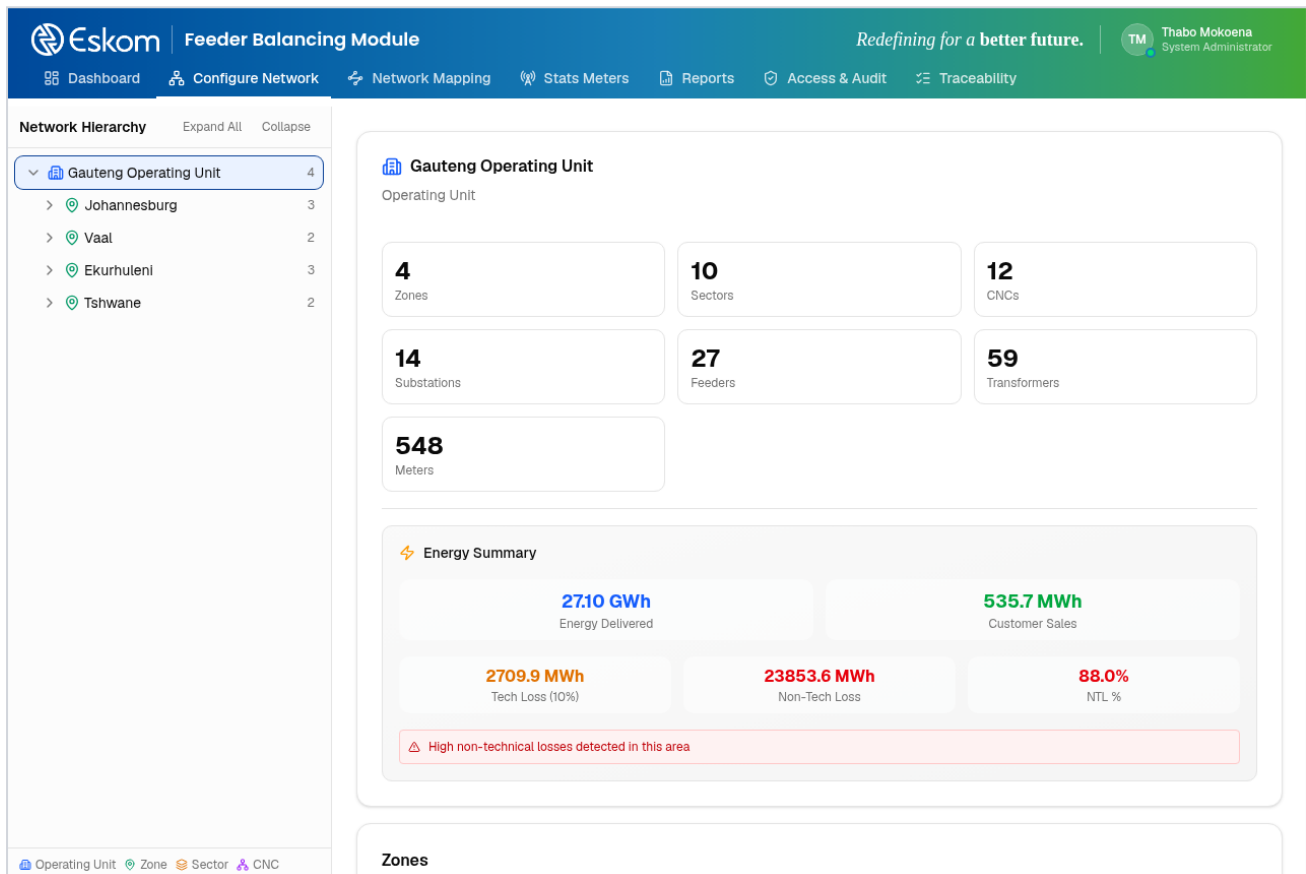
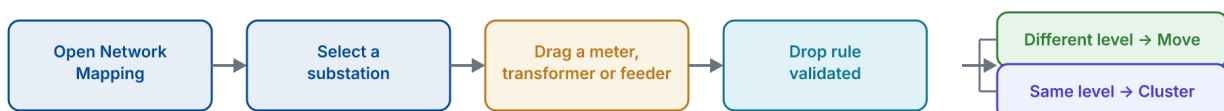


Figure 5.7 — Configure Network (live tool): the eight-level hierarchy with automatic energy roll-up and loss KPIs computed at every level.

Workflow — Network mapping (drag to reorganise)



Drops are validated live; only same-substation, rule-compliant moves are accepted.

Figure 5.5 — Network mapping workflow

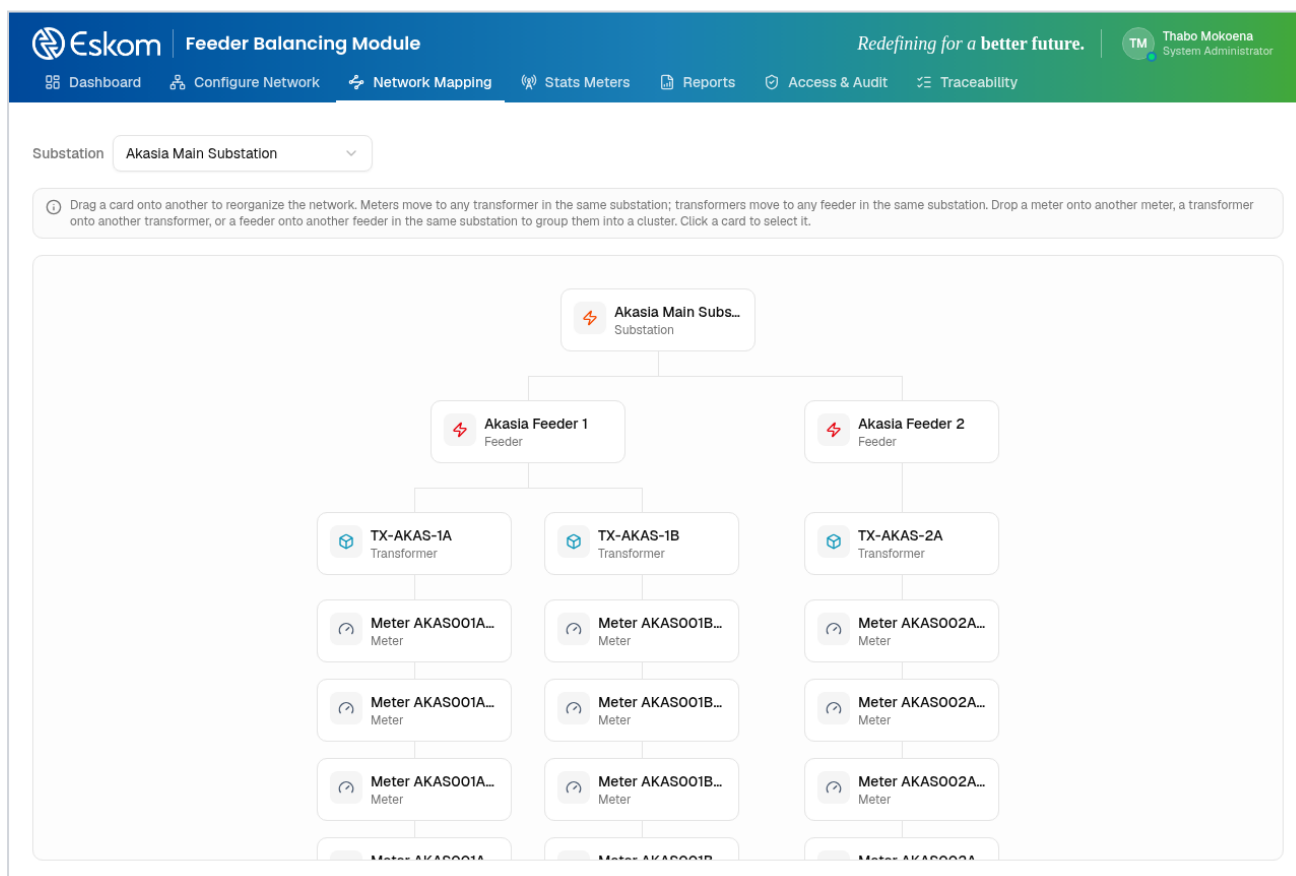


Figure 5.8 — Network Mapping (live tool): drag-and-drop organisation chart for a selected substation.

Workflow — Creating a meter cluster

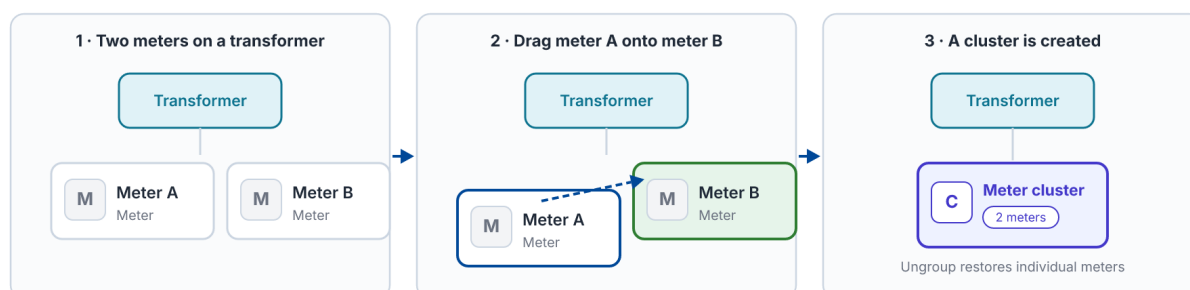


Figure 5.6 — Creating a meter cluster

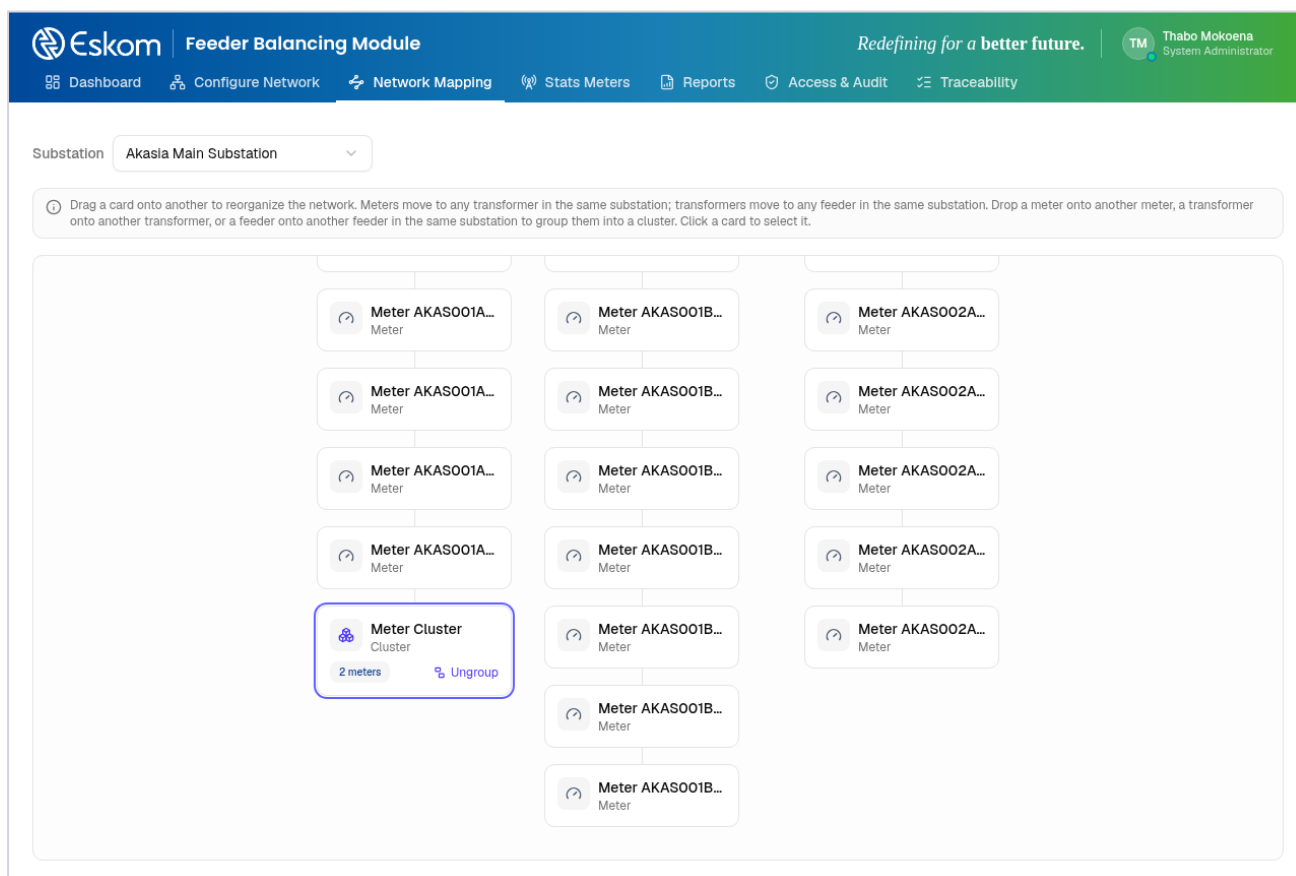


Figure 5.9 — Network Mapping (live tool): two meters grouped into a meter cluster, with one-click Ungroup to reverse the grouping.

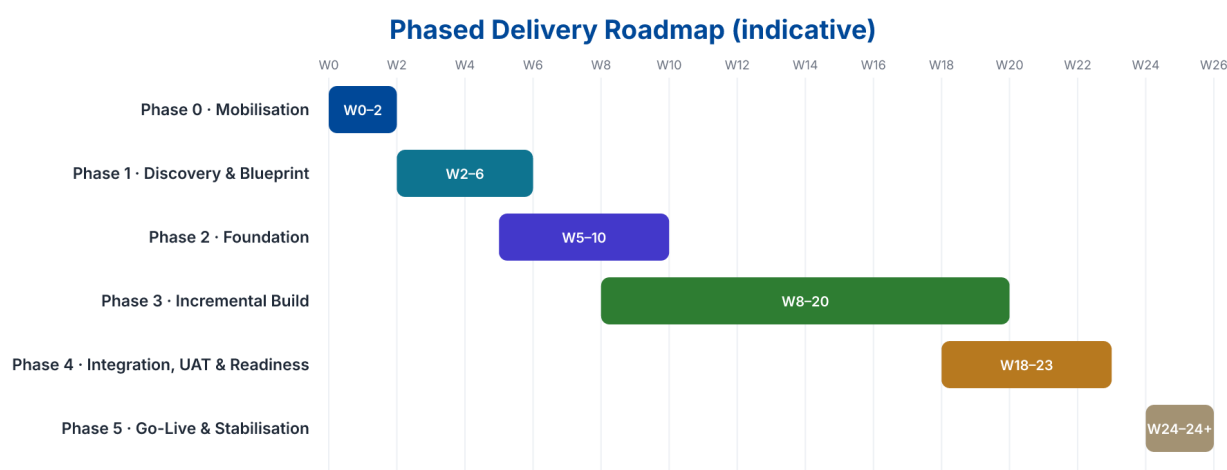
6. Delivery Methodology and Implementation Plan

Owned by Thabiso

T2 Technologies proposes a phased agile delivery model governed by formal stage gates. This balances iterative product development with the traceability and accountability expected in enterprise and public-sector programmes.

Phase	Indicative Timing	Key Activities	Exit Outcome
Phase 0 - Mobilisation	Weeks 1-2	Contract mobilisation, governance, access, stakeholder map, delivery plan, risks, environments and document baseline.	Approved mobilisation pack and detailed project plan.
Phase 1 - Discovery & Blueprint	Weeks 2-6	Current-state assessment, requirements, process mapping, architecture, security assessment and prioritisation.	Requirements baseline, target architecture, backlog and migration approach.
Phase 2 - Foundation	Weeks 5-10	Core platform, CI/CD, security baseline, integration foundation, environments and observability.	Operational technical foundation ready for solution increments.
Phase 3 - Incremental Build	Weeks 8-20	Applications, APIs, integrations, workflows, data pipelines and automated tests delivered in sprints.	Demonstrable functional releases with traceability.
Phase 4 - Integration, UAT & Readiness	Weeks 18-23	End-to-end testing, performance/ security testing, user acceptance, training, migration rehearsal and operational readiness.	UAT sign-off and go-live readiness approval.
Phase 5 - Go-Live & Stabilisation	Weeks 24+	Production deployment, hypercare, defect resolution, monitoring, support transition and post-implementation review.	Stable production service and transition to BAU support.

The above timeline is illustrative. The volume, data migration complexity,



Illustrative only — final schedule reconciles to issued tender milestones, scope volume and dependencies.

Figure 6.1 — Delivery roadmap

Sprint Delivery Cycle



Figure 6.2 — Sprint cycle

6.1 Sprint Delivery

- Prioritised product backlog linked to approved requirements.
- Regular planning, refinement, demonstrations and retrospectives.
- Definition of Done covering functional behaviour, testing, security, documentation and deployability.
- Formal change control for scope or contractual impacts outside the approved backlog baseline.
- Release notes and auditable evidence for each production release.

7. Programme Governance and Quality Management

Owned by Thabiso

Governance will provide clear authority, reporting, escalation and evidence-based decision making. T2 Technologies recommends a joint structure that includes executive oversight, programme management and technical working groups.

Forum	Purpose	Indicative Cadence
Steering Committee	Executive decisions, scope/budget oversight, strategic risks, escalations and milestone approval.	Monthly or as prescribed
Programme Review	Progress, dependencies, RAID log, commercial status, upcoming milestones and actions.	Weekly
Architecture / Security Review	Design decisions, integration standards, security controls, technical debt and exceptions.	Weekly / per major design
Sprint Ceremonies	Planning, stand-ups, demonstrations, retrospectives and backlog refinement.	Per sprint
Service Review	SLA performance, incidents, problems, capacity, releases and continuous improvement.	Monthly post go-live



Clear authority, reporting, escalation paths and evidence-based decision making across all forums.

Figure 7.1 — Governance structure

7.1 Quality Controls

- Requirements-to-test traceability.
- Peer review and controlled source-code management.
- Automated unit/integration testing wherever practical.
- Security checks integrated into the development lifecycle.
- Controlled environments and release approvals.
- Document versioning and approval records.
- Defect prioritisation and transparent closure evidence.
- Post-release verification and service health monitoring.

17. Commercial Proposal Framework / Pricing Schedule

Shared — Tumelo / Thabiso

Pricing must be completed strictly in the format prescribed by the tender. The following table can be used as an internal reconciliation schedule before values are transferred to the official pricing forms.

Cost Component	Pricing Basis	Amount (Excl. VAT)	Notes
Mobilisation & Discovery	Fixed price / milestone	R [INSERT]	
Architecture & Design	Fixed price / milestone	R [INSERT]	
Application & Integration Delivery	Fixed / sprint / deliverable	R [INSERT]	
Data & Migration	Fixed / estimated volume	R [INSERT]	
Cloud / DevOps Setup	Fixed price	R [INSERT]	Third-party cloud fees separate unless included
Testing & Go-Live	Fixed price	R [INSERT]	
Training & Knowledge Transfer	Fixed price	R [INSERT]	
Support & Maintenance	Monthly / annual	R [INSERT]	State SLA and support window
Third-Party Costs	At cost / specified	R [INSERT]	List separately
Total Tendered Price	As prescribed	R [INSERT]	Reconcile to official tender schedule

Commercial validity, escalation, travel guarantees and any exchange-rate